PARTICIPANT WORKBOOK

PostgreSQL Database Fundamentals with Performance & Tuning

Organised by A UK Trade and Infrastructure Enterprise Limited (AUKTIE)
Coventry Conferences, The Techno Centre, Coventry University Technology Park
Puma Way, Coventry CV1 2TT, United Kingdom | 22nd June – 3rd July 2026

Lead Instructor & Author

Dr Tertsegha Joseph Anande

PhD, MSc, MBCS, FHEA

Lecturer in Computing & Deputy Programme Leader, QAHE Limited / Ulster University — Birmingham, UK

Specialist in Machine Learning, AutoML, Ensemble Learning, Database Systems, and Cyber-Physical Security. Dr Anande brings both academic research rigour and industry-facing teaching practice to this programme, combining PostgreSQL database engineering with the data governance and digital transformation context required for national-scale social protection systems.

Profile & publications: drtertseghaanande.com
LinkedIn: linkedin.com/in/tertsegha-anande-phd-msc-fhea-mbcs-71939520a

Participant name:

Organisation / Ministry:

Role / Position:

Instructor note

REVISED SCHEDULE: most participants arrive in the UK the evening of Monday 22 June. Monday 22 June has been REMOVED from the programme. The programme now begins on TUESDAY 23 JUNE. Participants should arrive and settle in by 09:30; training begins at 10:00.

Monday 29 June is a Mid-Training Assessment Day — practical tasks only, no new teaching content. The Coventry City Council study visit has moved from Thursday 25 June to Friday 26 June.

Programme theme

Strengthening Enterprise Data Management, Digital Government Systems and Social Protection Delivery through PostgreSQL and International Best Practice

Intermediate learners: Focus on core syntax, concepts and guided exercises.	**Advanced learners:** Add challenge tasks, tuning rationale, and architectural design decisions.

Day	Theme / activity	Core skills you will build
W1 Mon 22 Jun	REMOVED — Programme now begins on Tuesday 23 June	Get PostgreSQL installed early if available; no content missed if you can't attend
W1 Tue 23 Jun	Foundations + Database Design & Architecture (merged)	Connect to PostgreSQL; normalise data; design NSR schema; explain WAL and MVCC
W1 Wed 24 Jun	SQL Fundamentals & Advanced Querying	Write DML/DDL; master JOINs; use window functions for analytics
W1 Thu 25 Jun	PostgreSQL Administration, Security & Backup	Implement RBAC; configure RLS; perform backup and recovery
W1 Fri 26 Jun	Study Visit — Coventry City Council	Observe local government digital transformation and citizen services
W2 Mon 29 Jun	Mid-Training Assessment Day	Demonstrate independent command of Tue/Wed/Thu material
W2 Tue 30 Jun	Performance Tuning & Database Optimisation	Read EXPLAIN plans; design indexes; partition large tables; monitor performance
W2 Wed 1 Jul	ETL/ELT & Data Governance (merged)	Design migration pipelines; map NDP Act to controls; design interoperability framework
W2 Fri 3 Jul	Programme Review & Closing Ceremony	Present your group's blueprint; reflect; receive your certificate

BEFORE YOU BEGIN

Complete this setup guide before Monday 22 / Tuesday 23 June

Instructor note

This setup guide must be completed on your own laptop before the programme begins. It takes 30-45 minutes. If you can attend Monday 22 June, that is a good time to do it. If not, arrive at 09:00 Tuesday 23 June — the day opens with a setup-verification slot built specifically for anyone arriving fresh.

PRE-SESSION SETUP GUIDE

Software installation through your first psql connection — complete before Day 1 (Tue 23 June)

Note

Complete this guide on your own laptop BEFORE the programme begins on Tuesday 23 June. Participants settle in from 09:30; training starts at 10:00.

It takes approximately 30-45 minutes depending on your internet connection.

If you get stuck, arrive by 09:30 for registration (settle-in from 09:30, training starts 10:00) for assisted setup.

No prior database experience is required.

Which PostgreSQL version should I install?

This programme is written against PostgreSQL 18 — the current stable release — and all exercises are tested against it. If you can install PostgreSQL 18, do so.

If you already have PostgreSQL 15, 16, or 17 installed on your laptop (for example, from previous work or another course), you do NOT need to uninstall it.
Every exercise in this programme runs without modification on versions 15-18.
Wherever a step is version-specific, this guide shows the PG18 command first, followed by an 'If you are on PG15-17' note where anything differs.

The only features that are PG18-only are explicitly marked 'PG18+' in the instructor guide — these are optional 'what's new' demonstrations, not required exercises.

What you will install

Software	What it is	Why you need it
PostgreSQL 18 (or 15-17)	The database server itself	The actual database engine used throughout the programme
psql	Command-line client (installed with PostgreSQL)	The primary tool used in every session
pgAdmin 4	Graphical interface (bundled on Windows)	Optional visual tool for browsing tables and data

Overview of the process

1. Install PostgreSQL 18 (includes psql and pgAdmin 4) — or use an existing 15-17 install
2. Verify the installation
3. Connect to PostgreSQL using psql for the first time
4. Create the training database and run your first command

Jump to the section for your operating system: Windows, macOS, or Ubuntu/Linux. All platforms then converge on the shared 'First connection' section.

WINDOWS 10 / 11 — INSTALLATION

1 Download the PostgreSQL 18 installer

5. Open your web browser and go to:

```
https://www.postgresql.org/download/windows/
```

6. Click 'Download the installer' — this takes you to the EDB download page
7. Find PostgreSQL 18.x (the latest 18.x version) for Windows x86-64 and download it (~300 MB)

Tip

If your internet connection is slow, ask the registration desk for a USB drive with an offline copy of the installer.

Already have PostgreSQL 15, 16, or 17 installed? You can skip this download entirely — your existing installation works for every exercise in this programme. Go straight to Step 5 to confirm your superuser password, then continue to 'First connection'.

2 Run the installer

8. Double-click the downloaded .exe file. If Windows shows a security warning, click Yes
9. Click Next on the welcome screen
10. Installation Directory: leave as default and click Next

3 Select components

Ensure ALL FOUR boxes are checked: PostgreSQL Server, pgAdmin 4, Stack Builder (can uncheck), Command Line Tools. Click Next.

4 Choose the data directory

11. Leave the default data directory and click Next

5 Set the superuser password — IMPORTANT

Important

For this training programme, set the password to: postgres

Use exactly this password (all lowercase). Every exercise assumes this password.
WRITE THIS DOWN — you will need it every time you connect.
(In production you would choose a strong unique password — 'postgres' is used here only to keep the training consistent for everyone.)

If you already have PostgreSQL 15-17 installed with a DIFFERENT password, either remember your existing password for use throughout the programme, or run: ALTER USER postgres PASSWORD 'postgres'; from psql to standardise it.

12. Type 'postgres' in the Password field, then again in Retype Password, then click Next

6 Set the port number

13. Leave the port as the default: 5432, click Next

Tip
If you already have an older PostgreSQL version installed and running on port 5432, the installer will suggest a different port (e.g. 5433) for the new instance.
Either is fine — note which port you used, as you will need it when connecting:
psql -U postgres -p 5433

7 Choose the locale

14. Leave the locale as default (or 'English, United Kingdom'), click Next

8 Complete the installation

15. Review the summary and click Next. Installation takes 2-5 minutes

Important
UNCHECK 'Launch Stack Builder at exit' before clicking Finish.
Stack Builder is not needed for this programme.

16. Click Finish

Checkpoint — confirm before continuing
PostgreSQL 18 (or your existing 15-17 installation) is ready.
pgAdmin 4 appears in your Start Menu.
Continue to 'First connection' below.

macOS — INSTALLATION

Works for both Intel and Apple Silicon (M1/M2/M3) Macs

Method A (Postgres.app) is simpler and recommended. Method B (Homebrew) is for existing Homebrew users.

Tip

Already have PostgreSQL 15, 16, or 17 installed and working? You do not need to upgrade — every exercise in this programme works on versions 15-18. Skip to Step 4 (Postgres.app) or the last command (Homebrew) below to confirm your superuser password, then go to 'First connection'. Just substitute your existing version number where '18' appears below.

Method A — Postgres.app (recommended)

1 Download and install

17. Go to https://postgresapp.com and click Download (~200 MB .dmg)
18. Double-click the .dmg, drag Postgres.app into Applications, then open it
19. If macOS blocks it: System Settings -> Privacy & Security -> 'Open Anyway'

2 Initialise the server

20. Click 'Initialize' or '+' to create a new server
21. This creates a PostgreSQL 18 server on port 5432 — the elephant icon appears in the menu bar

Tip

Postgres.app lets you run multiple versions side by side. If you already have a 15-17 server initialised, you can keep using that one — just note its port number (shown in the Postgres.app window) for the connection step.

3 Add psql to your PATH

22. Open Terminal (Cmd+Space, type 'Terminal') and run:

```
echo 'export PATH="/Applications/Postgres.app/Contents/Versions/18/bin:$PATH"' >>
~/.zshrc
```

Important

If you are using an existing PostgreSQL 15, 16, or 17 server instead, replace '18' in the path above with your version number, e.g. .../Versions/16/bin

23. Close and reopen Terminal, then verify:

```
psql --version
```

Tip

Expected: psql (PostgreSQL) 18.x (or 15.x/16.x/17.x if using an existing install) —
if 'command not found', restart your computer and retry.

4 Create the postgres superuser

```
psql -U $(whoami) -d postgres
CREATE ROLE postgres WITH SUPERUSER LOGIN PASSWORD 'postgres';
\q
```

Important

Use the training password: postgres — every exercise assumes this.
If this role already exists from a previous install, run instead:
ALTER ROLE postgres WITH PASSWORD 'postgres';

Method B — Homebrew (for existing Homebrew users)

```
brew install postgresql@18
brew services start postgresql@18
echo 'export PATH="/opt/homebrew/opt/postgresql@18/bin:$PATH"' >> ~/.zshrc
# restart Terminal, then:
psql --version
psql -d postgres -c "CREATE ROLE postgres WITH SUPERUSER LOGIN PASSWORD 'postgres';"
```

Tip

Already have postgresql@15, @16, or @17 via Homebrew? Just run:
brew services start postgresql@16 (substitute your version)
then the same psql --version and CREATE ROLE commands above.

Checkpoint — confirm before continuing

PostgreSQL is installed and running (version 18, or your existing 15-17).
Continue to 'First connection' below.

UBUNTU / DEBIAN LINUX — INSTALLATION

Tip
Already have postgresql-15, -16, or -17 installed and running? Skip to Step 4 to confirm the superuser password, then go to 'First connection'. Every exercise in this programme works on versions 15-18.

1 Add the PostgreSQL apt repository

```
sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" \
  > /etc/apt/sources.list.d/pgdg.list'
wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | \
  sudo gpg --dearmour -o /usr/share/keyrings/postgresql.gpg
sudo apt update
```

2 Install PostgreSQL 18 and pgAdmin

```
sudo apt install -y postgresql-18 postgresql-client-18
sudo apt install -y pgadmin4-desktop   # optional
```

Tip
To install an older supported version instead (e.g. if your distro's repo lags behind, or you prefer to match an existing production system), replace '18' with 15, 16, or 17 in the command above — e.g. postgresql-16 postgresql-client-16

3 Start the service

```
sudo systemctl start postgresql
sudo systemctl enable postgresql
sudo systemctl status postgresql   # look for 'active (running)', press q to exit
```

4 Set the postgres user password

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres';"
```

Important
Use the training password: postgres — every exercise assumes this.

Checkpoint — confirm before continuing
PostgreSQL (18, or your existing 15-17) is installed and running.
Continue to 'First connection' below.

FIRST CONNECTION — ALL OPERATING SYSTEMS

Verifying installation and connecting to psql for the first time

Operating system	How to open a terminal
Windows	Press the Windows key, type 'Command Prompt'. Or search for 'SQL Shell (psql)' for a pre-configured shortcut.
macOS	Press Cmd+Space, type 'Terminal', press Enter
Linux	Press Ctrl+Alt+T, or open Terminal from your applications menu

Step 1 — Connect to PostgreSQL

```
psql -U postgres
```

- If prompted 'Password for user postgres:', type postgres (characters won't be visible) and press Enter

Tip

Using 'SQL Shell (psql)' on Windows: press Enter to accept the four default prompts (Server/Database/Port/Username), then enter the password.

Expected result

```
postgres=#
```

Checkpoint — confirm before continuing

Seeing 'postgres=#' confirms you are connected — PostgreSQL is waiting for SQL commands.

Troubleshooting

Error message	Likely cause	Fix
'psql' is not recognized / command not found	psql not in PATH	Windows: use 'SQL Shell (psql)'. macOS: re-check PATH step. Restart terminal after PATH changes.
could not connect to server: Connection refused	Service not running	Windows: Services app -> start 'postgresql-x64-18' (or your version). macOS: open Postgres.app. Linux: sudo systemctl start postgresql
password authentication failed	Password not set to 'postgres'	Re-run the password-setting step for your OS
role 'postgres' does not exist	macOS: role not created	psql -U $(whoami) -d postgres, then CREATE ROLE postgres

		WITH SUPERUSER LOGIN PASSWORD 'postgres';

Step 2 — Run your first commands

Every SQL command ends with a semicolon (;). Commands starting with backslash (\) are psql meta-commands and do not need one.

2.1 — Check the PostgreSQL version

```
SELECT version();
```

Tip

Expected: PostgreSQL 18.x on x86_64-... — confirms PostgreSQL 18 is running.

If you are using an existing 15, 16, or 17 install, you will see that version number instead — for example 'PostgreSQL 16.4 on x86_64-...'. This is completely fine. Every exercise in this programme runs correctly on PostgreSQL 15 through 18. Make a note of YOUR version number — your instructor may ask for it during Day 1 so the room knows which version is in use at each station.

2.2 — List existing databases

```
\l
```

2.3 — Check your connection identity

```
SELECT current_user, current_database(), now();
```

Step 3 — Create the training database

```
CREATE DATABASE lab_nsr;
```

Tip

Expected: CREATE DATABASE

3.1 — Connect to it

```
\c lab_nsr
```

Tip

Your prompt changes from postgres=# to lab_nsr=#

3.2 — Install extensions used during the programme

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pgstattuple;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Important

If pgaudit or pg_stat_statements error mentioning 'shared_preload_libraries', don't worry — this needs a config change + restart, demonstrated in the Week 1 Friday session. Skip for now. pgcrypto and pgstattuple should install cleanly on all platforms.

3.3 — Verify

```
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

Step 4 — Exit psql

```
\q
```

Final checklist — confirm before Tuesday

#	Item	Done?
1	PostgreSQL is installed (version 18 recommended; 15-17 also fully supported)	
2	I can run 'psql -U postgres' and connect with password 'postgres'	
3	SELECT version(); returns PostgreSQL 15.x, 16.x, 17.x, or 18.x — I have noted which	
4	The lab_nsr database has been created	
5	I can connect to it with \c lab_nsr	
6	pgcrypto and pgstattuple extensions are installed	
7	I know how to exit psql using \q	

Ready for Tuesday

If all seven items are checked, your environment is ready for the programme.
Bring your laptop charger — sessions run 10:00–17:00 with breaks.
If anything failed, arrive by 09:30 Tuesday for assisted setup.
Keep this guide accessible during Day 1. Training starts at 10:00; arrive by 09:30 to settle in.

WEEK 1
MON 22 JUN

Optional Day — Registration, Environment Setup & Orientation

This day is optional

Most participants arrive in the UK the evening of Monday 22 June and will not be here today. If you ARE able to attend: welcome! Use the time to get your PostgreSQL environment set up early and relax before Tuesday's intensive start.

If you CANNOT attend: you have not missed anything required. Everything covered today gets a brief recap on Tuesday morning, and Tuesday opens with a setup-verification slot for exactly this reason. Just arrive Tuesday ready to start.

If you are here today

There is no fixed timetable — this is a relaxed, informal day. Suggested use of your time:

24. Complete the Pre-Session Setup Guide (included in this workbook) — install PostgreSQL, connect via psql, create the lab_nsr database
25. Meet other participants and your instructor informally
26. Read through the programme objectives and theme on the inside cover
27. Ask any early questions about the programme structure

Environment setup — do this if you have time today

Completing this now means you can skip the Tuesday morning setup-verification slot entirely and move straight into content.

Step	Command to run	Expected result	Done?
1	psql -U postgres	postgres=# prompt	
2	SELECT version();	PostgreSQL 18.x (or 15.x/16.x/17.x — any is fine)	
3	CREATE DATABASE lab_nsr;	CREATE DATABASE	
4	\c lab_nsr	You are now connected to database lab_nsr	
5	CREATE EXTENSION pgcrypto;	CREATE EXTENSION	

A little context, if there's time

This is a light introduction only — everything here is revisited on Tuesday, so don't worry about taking detailed notes.

DPI Layer	What it provides	Nigeria example
Digital Identity	Unique identification of every citizen	NIN — National Identity Number

Payment Rail	Fast, low-cost digital payment infrastructure	CBN NIBSS
Data Exchange	Interoperable citizen data across government	NSR — National Social Register

What data management problem in your organisation are you most hoping to solve with PostgreSQL? (You can also answer this Tuesday if you're not here today.)

WEEK 1
TUE 23 JUN

Enterprise Data Transformation, PostgreSQL Foundations & Architecture

If you attended Monday's optional session

You may already have PostgreSQL installed. Tuesday opens with a quick setup-verification check — confirm your setup works, then you can move ahead while others catch up. If you are arriving fresh today, that's completely normal — the first 20 minutes of today are built around exactly that.

Instructor note

This day merges what was originally planned as two separate days (Foundations and Architecture) since Monday 22 June was dropped from the schedule. It is a long, dense day — work through it steadily and don't worry if you don't finish every exercise; the Mid-Training Assessment on 29 June is your opportunity to consolidate.

Programme objectives this day covers

- PostgreSQL Database Administration (#1)
- Enterprise Database Architecture (#3)
- Social Protection Systems (#9)
- National Social Register Modernisation (#11)
- Public Sector Digital Transformation (#12)

Day schedule

Time	Session
09:00–09:30	Registration, welcome and participant introductions
09:30–09:50	Programme orientation and theme overview
09:50–10:30	Global trends in digital government & Digital Public Infrastructure
10:30–11:15	Introduction to PostgreSQL & ecosystem overview
11:15–13:00	PostgreSQL architecture: process, memory, storage, WAL, MVCC
13:00–14:00	Lunch
14:00–17:00	Schema development workshop: normalisation, ER design, environment setup

Key concepts: Digital Public Infrastructure

DPI Layer	What it provides	Nigeria example	International example
Digital Identity	Unique identification of	NIN — National Identity	India Aadhaar (1.4

	every citizen	Number	billion records)
Payment Rail	Fast, low-cost digital payment infrastructure	CBN NIBSS	India UPI (10B transactions/month)
Data Exchange	Interoperable citizen data across government	NSR — National Social Register	UK Government Data Exchange

What data management problem in your organisation are you most hoping to solve with PostgreSQL?

PostgreSQL architecture reference

Component	Role	What happens to NSR data
Postmaster process	Supervisor — listens for connections	Spawns a backend process for each caseworker connection
Backend process	One per connection — runs your SQL	Executes household lookup, payment update, report queries
Shared buffers	In-memory cache (25% of RAM recommended)	Hot household records kept in RAM for fast access
WAL (pg_wal/)	Sequential safety log written before data changes	Guarantees no committed payment is lost in a crash
Autovacuum	Background cleaner — removes dead row versions	Keeps payments table lean after monthly batch updates

WAL and MVCC exercises

Exercise T1.1 — Observe WAL generation

```
-- Record WAL position before
SELECT pg_current_wal_lsn() AS before_lsn;

-- Make a change
INSERT INTO nsr.states VALUES ('ZZ','Test State');
DELETE FROM nsr.states WHERE state_code = 'ZZ';

-- Record WAL position after
SELECT pg_current_wal_lsn() AS after_lsn;
```

before_lsn: _____ after_lsn: _____ Why did the LSN increase?

Exercise T1.2 — Two-session MVCC visibility

Open TWO psql terminals. Follow the steps below in the order shown.

Step	Terminal A	Terminal B
1	BEGIN; SELECT pmt_score FROM nsr.households WHERE household_id = 1;	—
2	(hold transaction open — do not commit)	UPDATE nsr.households SET pmt_score = 95.0 WHERE household_id = 1;
3	SELECT pmt_score FROM nsr.households WHERE household_id = 1;	—
4	COMMIT;	—
5	SELECT pmt_score FROM nsr.households WHERE household_id = 1;	—

Step 3 result: _____ Step 5 result: _____ Explain why READ COMMITTED shows this behaviour:

Normalisation exercises

Exercise T1.3 — Identify normalisation violations

The flat table below has multiple normalisation problems. Identify at least five.

household_code	head_name	state	lga	ward	num_members	pmt_score	payment_jan	payment_feb
HH-001	Musa Abdullahi	Kano	Kano Municipal	Sharada Ward	4	22.5	5000	5000

HH-002	Aisha Mohammed	KANO	Kano Municipal	Sharada ward	5	31.2	5000	0
HH-001	Musa Abdullahi	kano state	Kano Mun.	Sharada Ward	4	22.5	5000	5000
HH-003	Emeka Okafor	Lagos	Ikeja	Allen	three	N/A	10000	10000

Five normalisation problems and the PostgreSQL solution for each:

Exercise T1.4 — Design a new NSR table

Design a table for enumerators — the field staff who capture household data. Include all appropriate columns, constraints, and a COMMENT ON TABLE.

Your CREATE TABLE statement:

Environment setup checklist

Step	Command to run	Expected result	Done?
1	psql -U postgres	postgres=# prompt	
2	SELECT version();	PostgreSQL 18.x (or 15.x/16.x/17.x — any is fine)	
3	CREATE DATABASE lab_nsr;	CREATE DATABASE	
4	\c lab_nsr	You are now connected to database lab_nsr	

5	CREATE EXTENSION pgaudit;	CREATE EXTENSION	
6	CREATE EXTENSION pgcrypto;	CREATE EXTENSION	
7	SELECT COUNT(*) FROM nsr.households;	500000	

Any setup issues encountered and how resolved:

Advanced challenge — schema design

Advanced challenge (intermediate+ learners)

Design a programme_eligibility table that records which programmes a household qualifies for.

Required: household_id (FK), programme_code (e.g. NSIO, GEEP, TRADERMONI), effective_from, effective_to (nullable = still active),

eligibility_basis (the rule that qualified them), verified_by, verified_at.

Add a CHECK constraint ensuring effective_to > effective_from when both are set.

Add a UNIQUE constraint preventing the same household from having overlapping periods for the same programme.

Your CREATE TABLE statement:

Day reflection

Write 3 NSR data problems you identified today. Next to each one, predict which PostgreSQL feature might solve it.

WEEK 1
WED 24 JUN

SQL Fundamentals & Advanced Querying

Programme objectives this day covers

- SQL Development & Query Optimisation (#2)
- Data-Driven Decision Making (#10)

SQL quick reference

Statement	Purpose	NSR example
SELECT … FROM … WHERE	Retrieve and filter rows	SELECT head_name FROM households WHERE pmt_score < 30
GROUP BY … HAVING	Aggregate and filter groups	Households per LGA where count > 100
INNER JOIN	Only matching rows from both tables	Households with their state name
LEFT JOIN	All rows from left + matching from right	All households + their last payment (NULL if none)
Subquery (IN)	Filter using results of another query	Households in states beginning with 'K'
Subquery (EXISTS)	Filter if any related row exists	Households with at least one NIN-verified member
Window (RANK)	Rank rows within a partition	Poverty rank within each state
Window (LAG)	Previous row value in ordered set	Previous month payment for trend analysis

SQL exercises

Exercise W1.1 — DML operations

```
-- EXERCISE W1.1a: INSERT a new household
-- Add a household with: code='HH-EXERCISE-001', head='Your Name', lga_id=1,
-- ward='Training Ward', num_members=4, pmt_score=25.000, enumerated_at=today



-- EXERCISE W1.1b: UPDATE it (correct the num_members to 6)


-- EXERCISE W1.1c: DELETE it
```

Were any of the INSERT/UPDATE values rejected by a constraint? Which one and why?

Exercise W1.2 — JOINs

```
-- EXERCISE W1.2a: List all active households with:
-- household_code, head_name, lga_name, state_name, pmt_score
-- Ordered by state_name then pmt_score ascending



-- EXERCISE W1.2b: For each state, show:
-- state_name, total active households, households with pmt_score < 33.33,
-- percentage in bottom tercile



-- EXERCISE W1.2c: Find all active households with NO processed payment ever
```

Which state has the highest percentage in the bottom poverty tercile?

Exercise W1.3 — Window functions

```
-- EXERCISE W1.3a: For each household show:
-- household_code, head_name, state_name, pmt_score,
-- RANK within their state (lowest PMT = rank 1),
-- which national quintile they are in (NTILE 5),
-- their national percentile (PERCENT_RANK x 100, rounded to 1dp)



-- EXERCISE W1.3b: Monthly payment compliance per state
-- For each state and each month, show compliance % and
-- the change from the previous month using LAG()
```

Which state showed the largest single-month improvement in payment compliance?

Advanced challenge

Advanced challenge (intermediate+ learners)

Using a CTE, identify households that meet ALL THREE of the following criteria:
(1) PMT score below 25 (extreme poverty)
(2) Have at least one child under 18 (dob > CURRENT_DATE - INTERVAL '18 years')
(3) Have not received a processed payment in the last 6 months
Show: household_code, head_name, state_name, pmt_score, and the date of their last payment.

Your CTE query:

How many households match all three criteria? Which state has the most?

WEEK 1
THU 25 JUN

PostgreSQL Administration, Security & Backup Management

Programme objectives this day covers

- PostgreSQL Database Administration (#1)
- Data Governance (#6)
- Database Security (#7)

NSR role model reference

Role	Access level	State restriction	NIN visible?	Payment write?	Every query logged?
nsr_dba	Full access — all tables, all states	None (BYPASSRLS)	Yes	Yes	Yes — pgAudit
nsr_supervisor	All data in assigned states	BYPASSRLS (supervisor sees all)	Yes	Yes — UPDATE	Yes — session audit
nsr_caseworker	Household + individual read only	Own state only (RLS enforced)	Masked — last 4 chars	No	No (object audit only)
nsr_viewer	Aggregate views only — no individual rows	No individual rows visible	No	No	No
nsr_auditor	Read all tables across all states	None (full read)	Yes	No	Yes — every SELECT

Security lab exercises

Exercise H1.1 — Build the role model

```
-- EXERCISE H1.1: Create the five NSR roles
-- and two test login users per the role model reference above

-- Step 1: Create group roles (NOLOGIN)



-- Step 2: Set up role inheritance


-- Step 3: Grant table privileges



-- Step 4: Create test login users (password: 'NSR2026!')
```

```
-- Step 5: Verify with \du+
```

Paste the \du+ output showing all NSR roles and their membership:

Exercise H1.2 — Enable and test Row-Level Security

```
-- Step 1: Enable RLS
ALTER TABLE nsr.households ENABLE ROW LEVEL SECURITY;

-- Step 2: Create the policy (write it yourself based on the model)
-- Policy: caseworkers see only households in their app.state_code




-- Step 3: Test with Kano
SET ROLE nsr_caseworker;
SET app.state_code = 'KN';
SELECT COUNT(*) FROM nsr.households;

-- Step 4: Test with Lagos
SET app.state_code = 'LA';
SELECT COUNT(*) FROM nsr.households;

-- Step 5: Test with NO state set (unset variable)
RESET app.state_code;
SELECT COUNT(*) FROM nsr.households;

RESET ROLE;
```

Kano count: ____ Lagos count: ____ No state count: ____ Why does 'no state' return 0 not all rows?

Exercise H1.3 — Test the audit log trigger

```
-- The audit trigger was created in the session demo.
-- Make a change and query the audit log.

UPDATE nsr.households SET pmt_score = 28.5 WHERE household_id = 5;

SELECT log_id, event_time, db_user, operation, table_name,
       record_id,
       old_data->>'pmt_score' AS old_score,
       new_data->>'pmt_score' AS new_score
FROM   nsr.audit_log
WHERE  table_name = 'households'
ORDER  BY event_time DESC LIMIT 5;
```

```
-- Try to delete the audit record
DELETE FROM nsr.audit_log WHERE log_id = 1;
-- Expected: ERROR: permission denied
```

Paste the audit log row. What 6 pieces of information does it capture?

Backup exercises

Exercise H1.4 — Perform and verify a backup

```
-- Run in bash (not psql)
-- Step 1: Create a logical backup of the nsr schema
-- pg_dump -U postgres -d lab_nsr -n nsr -F c -f /tmp/nsr_backup.dump

-- Step 2: List the backup contents
-- pg_restore -l /tmp/nsr_backup.dump | head -30

-- Step 3: Create a test restore database
-- createdb -U postgres lab_nsr_test
-- pg_restore -U postgres -d lab_nsr_test -F c /tmp/nsr_backup.dump

-- Step 4: Verify restore in psql
-- \c lab_nsr_test
-- SELECT COUNT(*) FROM nsr.households;
-- Should match original count
```

Original row count: ____ Restored row count: ____ Do they match? YES / NO

NDP Act 2023 compliance mapping

NDP Act 2023 article	The control you implemented today	How to demonstrate compliance to an auditor
Art 2.5 — Lawful basis for every access		
Art 2.6 — Minimum necessary access		
Art 5.1 — Technical security measures		

WEEK 1
FRI 26 JUN

Study Visit 1 — Coventry City Council

Programme objectives this day covers

- Digital Welfare Administration (#8)
- Social Protection Systems (#9)
- Public Sector Digital Transformation (#12)

Theme

Local Government Digital Transformation & Citizen Services

Before you go — preparation

Write at least two specific questions you want answered today. Generic questions get generic answers.

My two specific questions for Coventry City Council:

Observation matrix — pick your focus area

Focus area	What to watch for
Technology	What database systems are in use? How is citizen identity managed and matched across services?
Governance	How are Subject Access Requests handled? What happens if there is a data breach?
Process	How do citizens access services — digitally vs in person? What does the journey look like end to end?
Policy	Which citizens are hardest to reach? What safety nets exist for those who fall through the cracks?

My assigned focus area today:

During the visit — notes

What did you observe? (be specific — systems, numbers, named processes)

Evening debrief — What / So What / Now What

WHAT did you observe that surprised you?

SO WHAT does it mean for how the NSR could work?

NOW WHAT — three lessons I will apply to my own organisation's practice:

WEEK 2
MON 29 JUN

Mid-Training Assessment Day

Important

This is a self-directed practical assessment, not a new teaching day.
Dr Anande is not in the room this morning and joins at 14:00.
A proctor is present from 10:00 to supervise the room, but cannot answer technical questions.
Work through Parts A, B, and C below at your own pace using your own PostgreSQL installation.
This is an individual task — please do not work with a neighbour today.
If you get stuck, write down where and why, then make your best attempt. Partial and incomplete answers are valuable — they show the instructor exactly where to focus this afternoon. There is no pass/fail mark; this is a checkpoint to help you, not a test against you.

Suggested timing

Time	Session
10:00–11:30	Part A — Foundations & Architecture
11:30–11:45	Break
11:45–13:00	Part B — SQL Fundamentals & Advanced Querying
13:00 (straight to lunch)	Part C — Administration, Security & Backup
13:00–14:00	Lunch
14:00–17:00	Dr Anande joins — live walkthrough and targeted refresher

Part A — Foundations & Architecture (from Tuesday 23 June)

A1 — Connection and version check

```
-- Connect to your training database and confirm your version
psql -U postgres -d lab_nsr
SELECT version();
```

Paste your version() output here:

A2 — Explain in your own words

In 3-4 sentences, explain what the Write-Ahead Log (WAL) is and why it matters for the NSR. Do not copy from the workbook — write it as you would explain it to a colleague.

A3 — Process model

```
-- Run this and explain what it shows
SELECT pid, usename, application_name, state
FROM   pg_stat_activity
ORDER  BY pid;
```

What does each row in this result represent? What is the difference between a backend process and the postmaster?

Part B — SQL Fundamentals & Advanced Querying (from Wednesday 24 June)

B1 — JOIN-based report

Write a query that lists, for each active household: household_code, head_name, state_name, lga_name, and pmt_score. Order by state then by pmt_score ascending.

Your SQL:

B2 — Window function

Write a query that ranks households by pmt_score within their own state (lowest score = rank 1), using a window function. Show household_code, state_name, pmt_score, and the rank.

Your SQL:

B3 — Households never paid

Write a query that finds all active households that have never received a processed payment. Use a JOIN — do not use a subquery.

Your SQL:

Part C — Administration, Security & Backup (from Thursday 25 June)

C1 — Create a restricted role

Create a role called assessment_viewer that can SELECT from nsr.households and nsr.lgas only — no other tables, and no write access of any kind.

Your SQL (CREATE ROLE and GRANT statements):

C2 — Backup approach

A colleague asks: 'If our PostgreSQL server crashes at 3am during the monthly payment run, how do we make sure no payment is lost?' Answer in 3-4 sentences referring to WAL and backup strategy.

Your answer:

C3 — Row-Level Security scenario

A Lagos caseworker should never be able to see Kano household records, even if the application has a bug. Name the PostgreSQL feature that enforces this at the database level, and briefly explain how it works.

Your answer:

End of self-directed session

Before you break for lunch

Make sure your workbook pages above are saved or handed to the proctor as instructed. Dr Anande will review everyone's answers and run a live walkthrough this afternoon, starting at 14:00. Bring any specific error messages or sticking points with you.

WEEK 2
TUE 30 JUN

Performance Tuning & Database Optimisation

Programme objectives this day covers

- SQL Development & Query Optimisation (#2)
- Performance Tuning (#4)

EXPLAIN ANALYZE key: reading the plan

Plan element	What it means	Good sign	Warning sign
Seq Scan	Reading all rows in a table	Acceptable if table is small	Slow on tables > 10k rows with selective WHERE
Index Scan	Using an index to jump to matching rows	Usually fast	Slow if many rows match (low selectivity)
Index Only Scan	All data from index; no table access	Fastest possible	Requires VACUUM to clear visibility map
actual time vs estimated	Actual vs planner estimate	Close numbers	Large gap = stale statistics, run ANALYZE
Buffers: hit	Pages served from shared_buffers cache	High number = good caching	Low = increase shared_buffers
Buffers: read	Pages read from disk	Low = good	High = cache too small or cold start

Performance tuning exercises

Exercise P1.1 — Diagnose and fix three slow queries

For each query: (1) time it with \timing on, (2) run EXPLAIN (ANALYZE, BUFFERS), (3) identify the problem, (4) create the appropriate index, (5) re-run and record improvement.

```
\timing on

-- Query A: Look up households for a specific NIN
SELECT h.household_code, h.head_name, h.pmt_score
FROM   nsr.households h
JOIN   nsr.individuals i ON i.household_id = h.household_id
WHERE  i.nin = 'AB1234567CD';

-- Query B: All active households with pmt_score < 30
SELECT household_code, head_name, pmt_score
FROM   nsr.households
WHERE  pmt_score < 30 AND is_active = TRUE
ORDER  BY pmt_score ASC;

-- Query C: Payment compliance by month for one state
SELECT DATE_TRUNC('month', p.payment_month)::DATE AS month,
       COUNT(*) FILTER (WHERE p.status = 'processed') AS paid,
       COUNT(*) AS total
```

```
FROM   nsr.payments p
JOIN   nsr.households h ON h.household_id = p.household_id
JOIN   nsr.lgas l ON l.lga_id = h.lga_id
WHERE  l.state_code = 'KN'
GROUP  BY DATE_TRUNC('month', p.payment_month);
```

Query	Before (ms)	EXPLAIN problem node	Index created	After (ms)	Improvement
A					
B					
C					

Exercise P1.2 — Design a partitioning strategy

The NSR payments table currently holds 2M rows (training dataset). At 1.2M payments per month for 5 years it will hold 72M rows. Design a partitioning strategy.

Your partitioning strategy: partition key, partition type, number of initial partitions, naming convention:

Write the CREATE TABLE statement for the partitioned table and two partitions:

Exercise P1.3 — Performance monitoring dashboard

```
-- Run all four monitoring queries and record the results

-- 1. Cache hit ratio
SELECT ROUND(100.0*SUM(heap_blks_hit)
/NULLIF(SUM(heap_blks_hit)+SUM(heap_blks_read),0),2) AS cache_hit_pct
FROM pg_statio_user_tables WHERE schemaname='nsr';

-- 2. Table bloat
SELECT relname, n_dead_tup, ROUND(n_dead_tup*100.0
/NULLIF(n_live_tup+n_dead_tup,0),1) AS pct_dead
FROM pg_stat_user_tables WHERE schemaname='nsr' ORDER BY n_dead_tup DESC;
```

```
-- 3. Unused indexes
SELECT indexname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes WHERE schemaname='nsr' ORDER BY idx_scan ASC LIMIT 5;

-- 4. Top 5 slowest queries
SELECT LEFT(query,60), calls, ROUND((total_exec_time/calls)::NUMERIC,1) AS avg_ms
FROM pg_stat_statements WHERE calls>5 ORDER BY avg_ms DESC LIMIT 5;
```

cache_hit_pct: ____	Is this above the 99% target?	Which table has most dead tuples?

Advanced challenge

Advanced challenge (intermediate+ learners)

For the payments table, create a COVERING INDEX that serves the following query with an Index Only Scan:

SELECT payment_month, amount_ngn, status FROM nsr.payments

WHERE household_id = $1 ORDER BY payment_month DESC;

Verify with EXPLAIN that Index Only Scan appears. Explain why 'index only' is faster than 'index scan'.

WEEK 2
WED 1 JUL

ETL/ELT, Data Migration & Data Governance

Instructor note

This day merges what was originally planned as two separate days (ETL/ELT and Governance), to make room for Monday's Mid-Training Assessment. This is the final technical day of the programme — the future-state architecture workshop this afternoon is your main deliverable, so pace yourself through the morning exercises.

Programme objectives this day covers

- ETL/ELT Processes (#5)
- Data Governance (#6)
- Digital Welfare Administration (#8)
- Data-Driven Decision Making (#10)
- National Social Register Modernisation (#11)
- Public Sector Digital Transformation (#12)

Part 1 — ETL/ELT & Data Migration (morning)

ETL vs ELT decision guide

Question	Choose ETL if...	Choose ELT if...
Where is transformation logic?	In an external tool (Talend, SSIS)	Inside PostgreSQL using SQL
Who maintains the pipeline?	Dedicated ETL developer team	SQL-literate DBA/data team
NSR recommendation?	Legacy integration to existing Oracle NSR	New PostgreSQL NSR — use ELT with dbt or stored procedures

ETL/ELT exercises

Exercise E1.1 — Data profiling

```
-- Profile: basic row counts and nullability
SELECT
  COUNT(*)                                          AS total_rows,
  COUNT(DISTINCT household_code)                   AS unique_codes,
  COUNT(*) - COUNT(DISTINCT household_code)        AS duplicate_codes,
  COUNT(*) FILTER (WHERE pmt_score ~ '^[0-9.]+$') AS numeric_scores
FROM nsr_staging.households_raw;
```

total_rows: ____ unique_codes: ____ duplicates: ____ invalid scores: ____

Exercise E1.2 — Deduplication and migration

```
-- Write the deduplication INSERT that:
-- 1. Takes only the most recently enumerated version of each household_code
-- 2. Only includes rows with valid num_members and known state
-- 3. Inserts into nsr.households with proper type conversions
-- 4. Uses ON CONFLICT DO UPDATE to handle existing records

WITH deduped AS (
  -- Step 1: add row_number to pick the most recent

)
INSERT INTO nsr.households (...)
SELECT ...
FROM   deduped d
JOIN   nsr.lgas l ON ...
WHERE  d.rn = 1
ON CONFLICT (household_code) DO UPDATE SET ...
;
```

How many rows were inserted? How many were updated (from ON CONFLICT)?

Exercise E1.3 — Reconciliation report

```
-- Write a reconciliation query showing:
-- staging_rows, valid_rows, loaded_rows, rejected_rows, load_rate_pct
```

load_rate_pct: ____ Is this acceptable? What threshold would you set for production migration?

Part 2 — Data Governance & NDP Act 2023 (late morning)

The six pillars of NSR data governance

Pillar	PostgreSQL control	Evidence for auditor
Data quality	CHECK constraints, triggers	Constraint list from pg_constraint
Access control	RBAC + Row-Level Security	EXPLAIN showing RLS filter in plan
Audit trail	pgAudit + audit_log trigger	audit_log rows with old/new values
Retention	Partitioning + pg_cron archive	Partition list showing dated partitions

Lineage	data_lineage table + staging logs	lineage rows for each entity
Interoperability	Views, FDW, logical replication	Working FDW or logical replication slot

NDP Act 2023 compliance exercise

For each NDP Act 2023 requirement below, identify which control you have implemented and write the SQL command that demonstrates it.

NDP Act 2023 Requirement	PostgreSQL control implemented	SQL command to demonstrate compliance
Art 2.5: Every data access must have a documented lawful basis and be attributable to a named person		
Art 2.6: Personnel must only access personal data that is necessary for their specific role		
Art 2.8: Data must not be kept longer than necessary for the purpose		
Art 5.1: Appropriate technical and organisational measures must protect personal data		

Metadata and lineage exercise

```
-- Query the data lineage for household_id = 42
SELECT source_system, migration_batch, ingested_at, ingested_by
FROM   nsr.data_lineage
WHERE  entity_type = 'household' AND entity_id = 42
ORDER  BY ingested_at;
```

How many lineage records exist for this household? Which source system is most recent?

Advanced challenge

Advanced challenge (intermediate+ learners)

Design a complete migration pipeline for NIN verification.
The NIMC provides a file with columns: nin, full_name, dob, gender, verified_at.
Your task: (1) load to staging, (2) validate NIN format, (3) match to existing individuals by nin,
(4) update nin_verified = TRUE and verified_at where matched,
(5) produce a reconciliation report showing match rate.

Your NIN verification pipeline SQL:

Part 3 — NSR future-state architecture workshop (afternoon)

This is your group's design deliverable for the programme closing ceremony. Complete the section assigned to your group.

My group: _____ **My role in the group:** _____

Group 1 — Data Layer Architecture

Design the PostgreSQL data architecture for the NSR at 20 million households.

Schema design: all tables, columns, and relationships:

Partitioning strategy with justification:

Index design: table, column, type, and justification for each index:

Group 2 — Security & Governance Architecture

Design the complete security and governance model for the NSR.

Role model: all roles with privileges, RLS policies, and use cases:

Audit architecture: what is logged, trigger design, retention policy:

NDP Act 2023 compliance matrix — map each article to a specific control:

Group 3 — Integration & Operations Architecture

Design the integration and operational architecture for the NSR.

ETL pipeline design: source systems, transformation steps, load method, frequency:

High-availability topology: node count, replication mode, failover mechanism, RTO/RPO:

Integration framework: NIN verification, CBN payments, state social registers:

Programme technical learning reflection

Topic	What I learned	How I will apply it in the NSR
Query optimisation & EXPLAIN		
Indexing & partitioning		
ETL/ELT pipeline design		
Data governance framework		
NDP Act 2023 compliance		
Interoperability frameworks		

My single most important learning from this programme's technical days:

One specific change I will implement in my organisation within 90 days of returning:

WEEK 2
FRI 3 JUL

Programme Review & Closing Ceremony

Today

This afternoon marks the formal close of the programme. You will watch (or present, if you are in a presenting group) the three future-state NSR architecture blueprints, reflect on the study visits, and receive your Certificate of Attendance.
Please complete the Reflection Sheet and Feedback Form below — both matter.

Running order

Time	Session
14:00–14:10	Welcome back
14:10–15:40	Group architecture blueprint presentations (Data Layer / Security & Governance / Integration)
15:40–15:55	Break
15:55–16:15	Study visit reflections — open discussion
16:15–16:30	Knowledge transfer discussion
16:30–16:45	Certificate presentation
16:45–16:55	Closing remarks
16:55 onward	Group photo & networking reception

Reflection sheet

Take a few quiet minutes to complete this before the knowledge transfer discussion. Specific answers are far more useful to you in three months than general ones.

Looking back

What is the single most useful technical skill you gained in these two weeks, and why that one specifically?

What is one moment from a study visit (Coventry City Council or others) that changed how you think about your own organisation?

What was the hardest concept to grasp, and what finally made it click?

Looking forward

Name one specific person in your organisation who needs to hear what you learned here, and what you will tell them first.

What is the SINGLE most important change you will personally make within 30 days of returning?

What would you need (support, resources, authority) to actually make that change happen?

Is there anything from this programme you want to keep learning about on your own? Where will you go to learn it?

Programme feedback form

Your honest feedback directly shapes future iterations of this programme. Please be specific where possible.

Rate each of the following (1 = poor, 5 = excellent)

Area	1	2	3	4	5
Technical content depth and relevance					
Pace of the programme overall					
Quality of practical exercises and labs					
Instructor clarity and approachability					
Study visit organisation and value					
Venue and logistics (Coventry Conferences)					
Pre-session setup guide and materials					
Mid-Training Assessment Day (29 June)					
Overall value relative to time invested					

Open feedback

What worked particularly well and should definitely be kept in future programmes?

What would you change about the programme structure, pace, or content?

Was the balance between technical training days and study visits right? Too much of one or the other?

Any other comments for AUKTIE or the Lead Instructor?

Name (optional): _____ Organisation: _____

Certificate of attendance — preview

Your physical certificate will be presented this afternoon. The information below confirms what it will state — please flag any spelling corrections to the registration desk before 16:30.

A UK TRADE AND INFRASTRUCTURE ENTERPRISE LIMITED (AUKTIE)

CERTIFICATE OF ATTENDANCE

This certifies that

_____

has successfully completed the

Executive Training & Study Visit Programme

PostgreSQL Database Fundamentals with Performance & Tuning

22 June – 3 July 2026 • Coventry, United Kingdom

_____ _____

Dr Tertsegha Joseph Anande, Lead Instructor AUKTIE Representative

Tip

This preview exists so any name-spelling errors are caught BEFORE printing. If you see an error above, find the registration desk immediately — corrections after 16:30 today cannot be guaranteed before the ceremony.